OBJECT ORIENTED PROGRAMMING (CDT2001)

UNIT - IV

Contents

- Exceptions
- Types of Exception
- Use of try catch block
- Handling multiple exceptions
- Using finally, throw and throws clause
- User defined exceptions
- Streams
- Byte streams, character streams
- File Handling
- Serialization

Exceptions

- Exceptions are abnormal conditions that arise at runtime causing Termination of a program.
- Exceptions are the runtime errors.
- An Exception is an unwanted event that interrupts the normal flow of the program.
- By handling the exceptions we can provide a meaningful message to the user about the issue rather than a system generated message, which may not be understandable to a user.
- Due to design errors or coding errors, our programs may fail in unexpected ways during execution.

Real Life Example

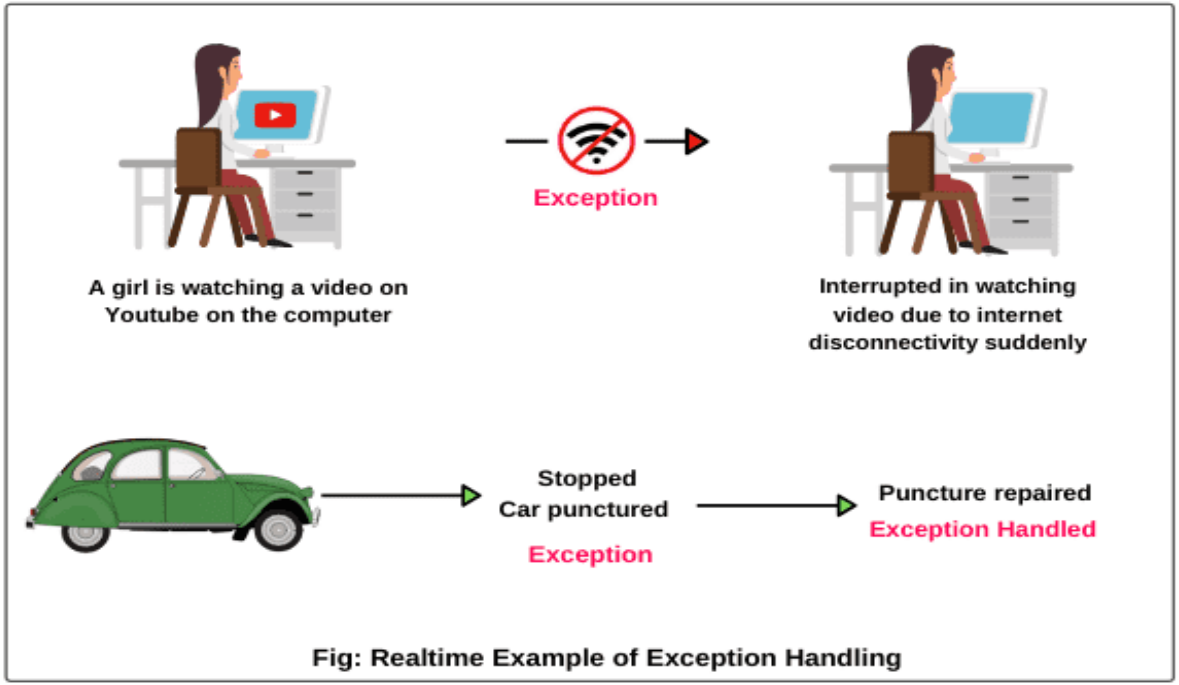


Fig: Realtime Example of Exception Handling

Error

- An error may produce an incorrect output or may terminate the execution of the program abruptly or even may cause the system to crash. So it is our responsibility to detect and manage the error properly.

Types of Errors:

- **Runtime Errors**: occur while the program is running if the environment detects an operation that is impossible to carry out.
- **Logic Errors:** occur when a program doesn't perform the way it was intended
- **Syntax Errors:** Arise because the rules of the language have not been followed. They are detected by the compiler.

Example of Run Time Error


```
Class Error
{
public static void main(String args[])
 {
int a=10;
int b=5;
int c=5;

int x=a/(b+c);
System.out.println("x=" +x);
int y=a/(b-c);   // Errorr division by zero
System.out.println("y=" +y);
}
}
```

Why an Exception occurs?

There can be several reasons that can cause a program to throw exception

For example: Opening a non-existing file in your program, Network connection problem, bad input data provided by user etc.

What happens when an exception occurs during runtime??

- The method causing exception creates an Exception object and hands it to the JVM
- This is called as Throwing an exception
- The exception object contains information about the exception that has occurred
- JVM looks for exception handler in the same method, if not found it looks back in the call stack until a handler is found
- If handler is found nowhere, the program finally terminates
- If handler is found then the program execution continues

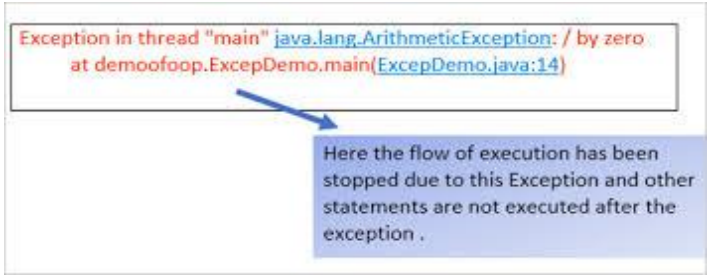
Example


```
public class Main{
 public static void main(String[] args) {
                                        m();
                                }
                                static void m(){
            n();
     }
    static void n(){
            int i=1;
            System.out.println(i/0);
}
}
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Main.n(Main.java:18)
        at Main.m(Main.java:14)
        at Main.main(Main.java:11)
```

Exception Handling

- Exception Handling is a mechanism to handle runtime errors
- Normal flow of the application can be maintained
- It is an object which is thrown at runtime
- Exception handling done with exception object
- If an exception occurs, which has not been handled by programmer then program execution gets terminated and a system generated error message is shown to the user



Advantage of exception handling

- Exception handling ensures that the flow of the program doesn’t break when an exception occurs
- For example, if a program has bunch of statements and an exception occurs mid way after executing certain statements then the statements after the exception will not execute and the program will terminate abruptly
- By handling we make sure that all the statements execute and the flow of program doesn’t break

Difference between error and exception

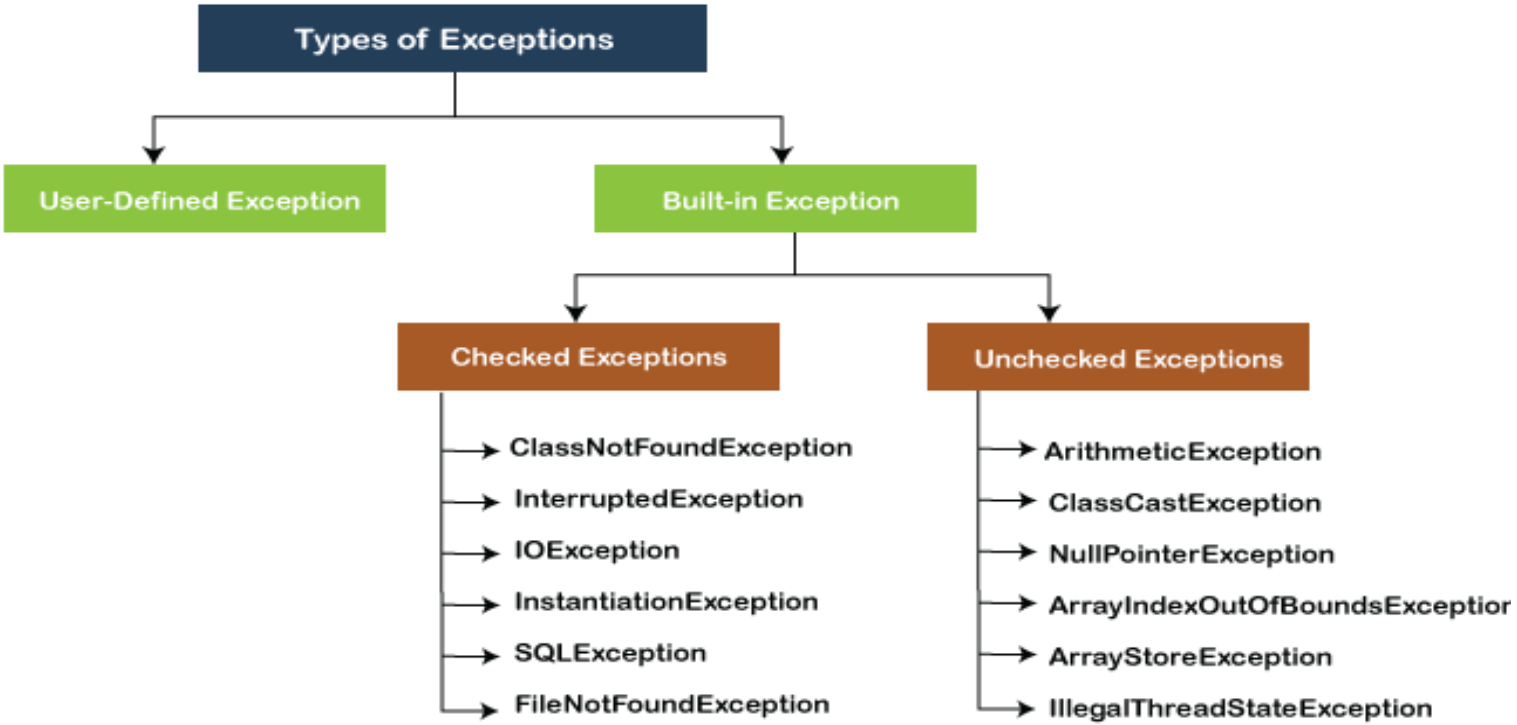
RCOEM
Shri Ramdeobaba College of Engineering and Management, Nagpur

Exception Handling

Differences Between Error & Exception

ERROR	EXCEPTION
in "java.lang.Error" package.	in "java.lang.Exception" package.
Lacking of system resources.	Because of the code.
It is impossible to recover.	It is possible to recover by handling "try-catch block"
classified as unchecked type.	classified as checked or unchecked type.
Examples : java.lang.StackOverflowError java.lang.OutOfMemoryError	Examples : Checked Exceptions : ClassNotFoundException Unchecked Exceptions : NullPointerException

Types of exceptions



Built In and User Defined Exceptions

- Built-in exceptions are the exception classes defined in the Java library
- User defined exception class is the exception that can be defined by a programmer if built in exceptions are not applicable

Checked vs. Unchecked Exceptions

- Checked Exceptions must be checked by programmer, otherwise program does not compile.
- The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time
- Unchecked exceptions need not be checked by programmer
- The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime

Some common Exceptions

Exception	Description	Typically Thrown
`ArrayIndexOutOfBoundsException` (this chapter)	Thrown when attempting to access an array with an invalid index value (either negative or beyond the length of the array).	By the JVM
`ClassCastException` (Chapter 2)	Thrown when attempting to cast a reference variable to a type that fails the IS-A test.	By the JVM
`IllegalArgumentException`	Thrown when a method receives an argument formatted differently than the method expects.	Programmatically
`IllegalStateException`	Thrown when the state of the environment doesn't match the operation being attempted—for example, using a scanner that's been closed.	Programmatically
`NullPointerException` (Chapter 3)	Thrown when attempting to invoke a method on, or access a property from, a reference variable whose current value is `null`.	By the JVM
`NumberFormatException` (this chapter)	Thrown when a method that converts a `String` to a number receives a `String` that it cannot convert.	Programmatically
`ArithmeticException`	Thrown when an illegal math operation (such as dividing by zero) is attempted.	By the JVM
`ExceptionInInitializerError` (Chapter 2)	Thrown when attempting to initialize a static variable or an initialization block.	By the JVM
`StackOverflowError` (this chapter)	Typically thrown when a method recurses too deeply. (Each invocation is added to the stack.)	By the JVM
`NoClassDefFoundError`	Thrown when the JVM can't find a class it needs, because of a command-line error, a classpath issue, or a missing `.class` file.	By the JVM

Keywords in Java Exception Handling

- try
- catch
- finally
- throw
- throws

Try Catch in Java – Exception handling

- The try block contains set of statements where an exception can occur
- A try block is always followed by a catch block, which handles the exception that occurs in associated try block
- A try block must be followed by catch blocks or finally block or both

Syntax of try block

```
try{
   //statements that may cause an exception
}
```

Catch block

- When an exception occurs in try block, the corresponding catch block that handles that particular exception executes.
- For example if an arithmetic exception occurs in try block then the statements enclosed in catch block for arithmetic exception executes

Syntax of try catch in java

```
try
{
    //statements that may cause an exception
}
catch (exception(type) e(object))
{
    //error handling code
}
```

Example: try catch block

- If an exception occurs in try block then the control of execution is passed to the corresponding catch block
- A single try block can have multiple catch blocks associated with it, you should place the catch blocks in such a way that the generic exception handler catch block is at the last
- The generic exception handler can handle all the exceptions but you should place it at the end, if you place it at the before all the catch blocks then it will display the generic message.
- You always want to give the user a meaningful message for each type of exception rather then a generic message

More Examples on try catch block

Example1: to resolve the exception in a catch block
Example2: along with try block, we also enclose exception code in a catch block
Example 3: NullPointerException

```
public class TryCatchExample1 {

    public static void main(String[] args) {
        int i=50;
        int j=0;
        int data;
        try
        {
        data=i/j; //may throw exception
        }
            // handling the exception
        catch(Exception e)
        {
             // resolving the exception in catch block
            System.out.println(i/(j+2));
        }
    }
```

```
public class TryCatchExample2 {


    public static void main(String[] args) {

        try
        {
        int data1=50/0; //may throw exception

        }
             // handling the exception
        catch(Exception e)
        {
            // generating the exception in catch block
        int data2=50/0; //may throw exception

        }
    System.out.println("rest of the code");
    }
}
```

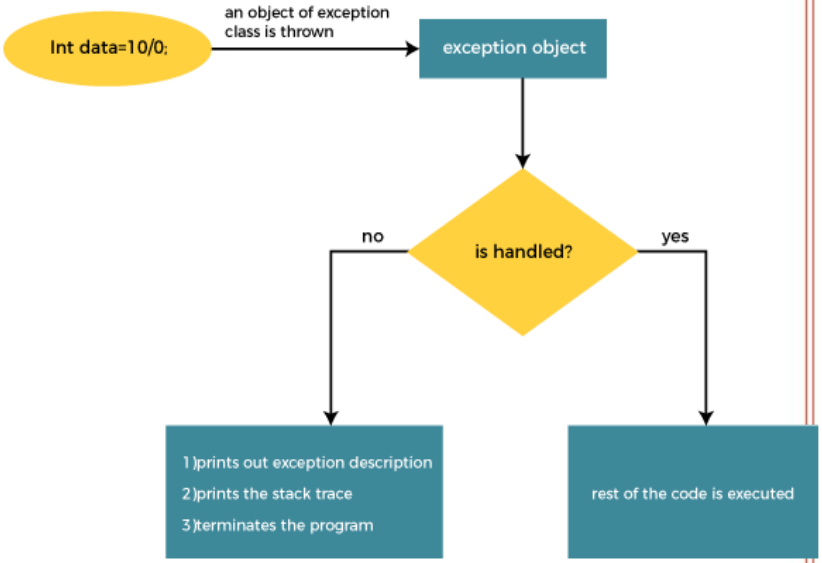
```
public class TryCatchExample3 {
	public static void main(String[] args) {
		try{
		 String s=null;
		 System.out.println(s.length());
		}
		catch(ArithmeticException e)
		  {
		   System.out.println("Arithmetic Exception occurs");
		  }
		catch(ArrayIndexOutOfBoundsException e)
		  {

System.out.println("ArrayIndexOutOfBounds Exception occurs");
		  }
 //catch(NullPointerException e)
		  //{
		//     System.out.println("NullPointerException has occurred");
		// }
		    catch(Exception e)
		      {
		       System.out.println("Parent Exception occurs");
		      }
		    System.out.println("rest of the code");
	}
}
```

Internal Working of Java try-catch block

The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

- Prints out exception description
- Prints the stack trace (Hierarchy of methods where the exception occurred)
- Causes the program to terminate
- But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed



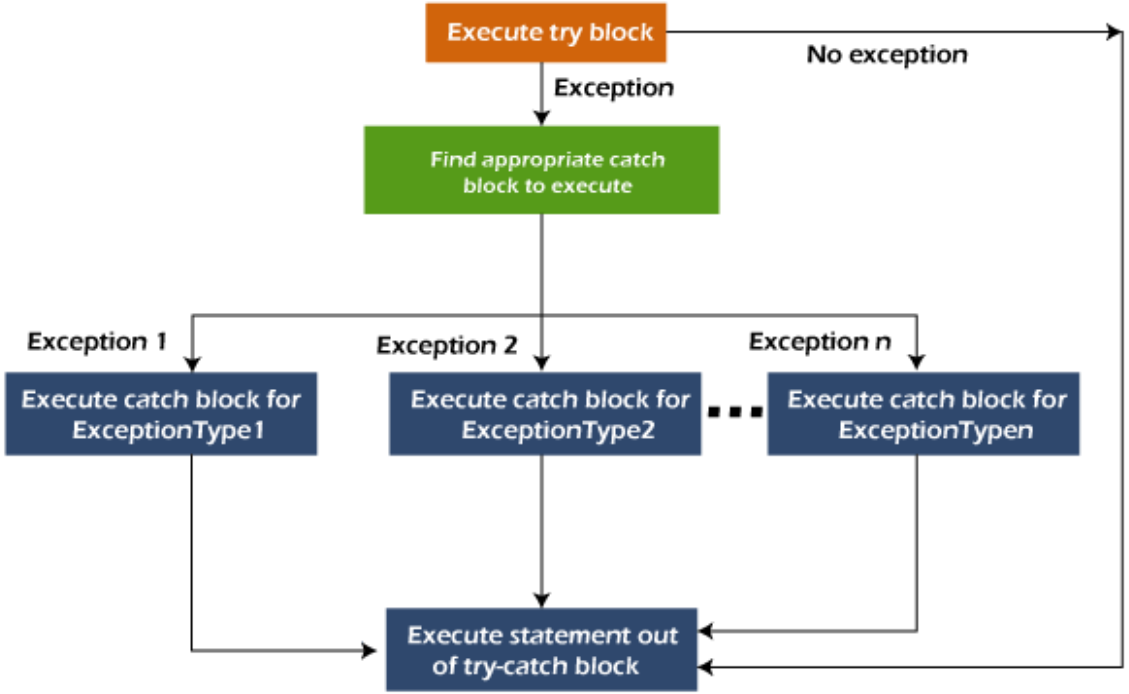
Multiple Catch Blocks

- In this example there are multiple catch blocks and these catch blocks executes sequentially when an exception occurs in try block.
- Which means if you put the last catch block (catch(Exception e)) at the first place, just after try block then in case of any exception this block will execute as it can handle all exceptions.
- This catch block should be placed at the last to avoid such situations.

```
public class MultipleCatchBlock {

  public static void main(String args[]){
    try{
        int a[]=new int[7];
        a[4]=30/0;
        System.out.println("First print statement in try block");
    }
    catch(ArithmeticException e){
       System.out.println("Warning: ArithmeticException");
    }
    catch(ArrayIndexOutOfBoundsException e){
       System.out.println("Warning: ArrayIndexOutOfBoundsException");
    }
    catch(Exception e){
       System.out.println("Warning: Some Other exception");
    }
   System.out.println("Out of try-catch block...");
  }
}
```

Working of Multiple Catch Blocks



Nested try catch block in Java – Exception handling

- When a try catch block is present in another try block then it is called the nested try catch block.
- Each time a try block does not have a catch handler for a particular exception, then the catch blocks of parent try block are inspected for that exception, if match is found that catch block executes.
- If neither catch block nor parent catch block handles exception then the system generated message would be shown for the exception, similar to what we see when we don’t handle exception.

Syntax of Nested try Catch


```
//Main try block
try {
   statement 1;
   statement 2;
   //try-catch block inside another try block
   try {
      statement 3;
      statement 4;
      //try-catch block inside nested try block
      try {
         statement 5;
         statement 6;
      }
      catch(Exception e2) {
            //Exception Message
      }
   }
   catch(Exception e1) {
      //Exception Message
   }
}
//Catch of Main(parent) try
block
catch(Exception e3) {
      //Exception Message
}
```

Nested Try Catch Example

- The ArrayIndexOutOfBoundsException occurred in the grand child try-block3
- Since try-block3 is not handling this exception, the control then gets transferred to the parent try-block2 and looked for the catch handlers in try-block2
- Since the try-block2 is also not handling that exception, the control gets transferred to the main (grand parent) try-block where it found the appropriate catch block for exception
- This is how the nesting structure works

```
public class NestedTryCatch {

  public static void main(String args[]){
    //main try-block
    try{
                //try-block2
      try{
        //try-block3
        try{
                int arr[]= {1,2,3,4};
                /* I'm trying to display the value of  an element
which doesn't exist. The
                 * code should throw an exception                */
                System.out.println(arr[10]);
        }
        catch(ArithmeticException e){
                System.out.print("Arithmetic Exception");
                System.out.println(" handled in try-block3");
        }
      }
catch(ArithmeticException e){
        System.out.print("Arithmetic Exception");
        System.out.println(" handled in try-block2");
      }
    }
    catch(ArithmeticException e3){
                System.out.print("Arithmetic Exception");
                System.out.println(" handled in main try-block");
    }
    catch(ArrayIndexOutOfBoundsException e4){

                System.out.print("ArrayIndexOutOfBoundsException"
);
                System.out.println(" handled in main try-block");
    }
    catch(Exception e5){
                System.out.print("Exception");
                System.out.println(" handled in main try-block");
     }
   }
}
```

Example 2: Nested try block

Block1: I have divided an integer by zero and it caused an ArithmeticException, since the catch of block1 is handling ArithmeticException "Exception: e1" displayed

Block2: In block2, ArithmeticException occurred but block 2 catch is only handling ArrayIndexOutOfBoundsException so in this case control jump to the Main try-catch(parent) body and checks for the ArithmeticException catch handler in parent catch blocks. Since catch of parent try block is handling this exception using generic Exception handler that handles all exceptions, the message “Inside parent try catch block” displayed as output

Parent try Catch block: No exception occurred here so the “Next statement..” displayed.

```
public class NestedTryCatch1 {

  public static void main(String args[]){
                  //Parent try block
   try{
                  //Child try block1
      try{
        System.out.println("Inside block1");
        int b =45/0;
        System.out.println(b);
      }
      catch(ArithmeticException e1){
        System.out.println("Exception: e1");
      }
      //Child try block2
      try{
        System.out.println("Inside block2");
        int b =45/0;
        System.out.println(b);
      }
      catch(ArrayIndexOutOfBoundsException e2){
            System.out.println("Exception: e2");
          }
         System.out.println("Just other statement");
    }
    catch(ArithmeticException e3){
                  System.out.println("Arithmetic Exception");
        System.out.println("Inside parent try catch block");
    }
    catch(ArrayIndexOutOfBoundsException e4){
                  System.out.println("ArrayIndexOutOfBoundsException");
        System.out.println("Inside parent try catch block");
    }
    catch(Exception e5){
                  System.out.println("Exception");
        System.out.println("Inside parent try catch block");
    }
    System.out.println("Next statement..");
  }
}
```

Important

NOTE: Whenever the child catch blocks are not handling any exception, it jumps to the parent catch blocks, if the exception is not handled there as well then the program will terminate abruptly showing system generated message.

Finally Block

- A finally block contains all the crucial statements that must be executed whether exception occurs or not
- The statements present in this block will always execute regardless of whether exception occurs in try block or not such as closing a connection, stream etc.
- Written at the last.. After try block or try-catch blocks.
- If an exception is thrown, finally runs. If an exception is not thrown, finally runs.
- If the exception is caught, finally runs. If the exception is not caught, finally runs.

Syntax of Finally block


```
try {
    //Statements that may cause an exception
}
catch {
   //Handling exception
}
finally {
   //Statements to be executed
}
```

A Simple Example of finally block

The exception occurred in try block which has been handled in catch block, after that finally block got executed

```
public class FinallyBlock {

   public static void main(String args[]) {
      try{
                  int num=121/0;
                  System.out.println(num);
      }
      catch(ArithmeticException e){
         System.out.println("Number should not be divided by zero");
      }
      /* Finally block will always execute
       * even if there is no exception in try block
       */
      finally{
                  System.out.println("This is finally block");
      }
      System.out.println("Out of try-catch-finally");
   }
}
```

Few Important points regarding finally block

- A finally block must be associated with a try block, you cannot use finally without a try block. You should place those statements in this block that must be executed always
- Finally block is optional, as we have seen in previous tutorials that a try-catch block is sufficient for exception handling, however if you place a finally block then it will always run after the execution of try block
- In normal case when there is no exception in try block then the finally block is executed after try block. However if an exception occurs then the catch block is executed before finally block
- An exception in the finally block, behaves exactly like any other exception
- The statements present in the finally block execute even if the try block contains control transfer statements like return, break or continue

Example of finally block and return statement

Even if return statement in the method, the finally block still runs

```
public class FinallyBlockReturn {

  public static void main(String args[])
  {
    System.out.println(FinallyBlockReturn.myMethod());
  }
  public static int myMethod()
  {
    try {
      return 112;
    }
    finally {
      System.out.println("This is Finally block");
      System.out.println("Finally block ran even after return statement");
    }
  }
}
```

Rule: For each try block there can be zero or more catch blocks, but only one finally block

Case 1: When an exception does not occur
Case 2: When an exception occur but not handled by the catch block
Case 3: When an exception occurs and is handled by the catch block

Flow of control in try/catch/finally blocks

- If exception occurs in try block's body then control immediately transferred(skipping rest of the statements in try block) to the catch block. Once catch block finished execution then finally block and after that rest of the program
- If there is no exception occurred in the code which is present in try block then first, the try block gets executed completely and then control gets transferred to finally block (skipping catch blocks)
- If a return statement is encountered either in try or catch block. In this case finally block runs. Control first jumps to finally and then it returned back to return statement

How to throw exception in java with example

- In Java we have already defined exception classes such as ArithmeticException, NullPointerException, ArrayIndexOutOfBounds exception etc. These exceptions are set to trigger on different-2 conditions. For example when we divide a number by zero, this triggers ArithmeticException, when we try to access the array element out of its bounds then we get ArrayIndexOutOfBoundsException

- We can define our own set of conditions or rules and throw an exception explicitly using throw keyword

- For example, we can throw ArithmeticException when we divide number by 5, or any other numbers, what we need to do is just set the condition and throw any exception using throw keyword

Throw Keyword

- Used to Explicitly throw an exception
- Can throw check or unchecked exception
- Mainly used to throw user defined exceptions
- The thrown exception must be an instance of Throwable or its subclass

Throw Keyword

Syntax of throw keyword:

throw new exception_class("error message");

For example:
throw new ArithmeticException("dividing a number by 5 is not allowed in this program");

Example of Throw keyword

- Lets say we have a requirement where we need to only register the students when their age is less than 12 and weight is less than 40, if any of the condition is not met then the user should get an ArithmeticException with the warning message “Student is not eligible for registration”

- We have implemented the logic by placing the code in the method that checks student eligibility if the entered student age and weight doesn’t met the criteria then we throw the exception using throw keyword

```
public class ThrowExample1 {
    /* In this program we are checking the Student age
 * if the student age<12 and weight <40 then our program
 * should return that the student is not eligible for registration.
 */

   static void checkEligibilty(int stuage, int stuweight){
      if(stuage<12 && stuweight<40) {
         throw new ArithmeticException("Student is not eligible for registration");
      }
      else {
         System.out.println("Student Entry is Valid!!");
      }
   }
public static void main(String args[]){
    System.out.println("Welcome to the Registration process!!");
    checkEligibilty(10, 39);
    System.out.println("Have a nice day..");
 }
}
```

Throwing UnChecked Exception

- In this example, we have created a method named validate() that accepts an integer as a parameter
- If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote
- Note: If we throw unchecked exception from a method, it is must to handle the exception or declare in throws clause

```
public class ThrowUnchecked {

    //function to check if person is eligible to vote or not
    public static void validate(int age) {
        if(age<18) {
            //throw Arithmetic exception if not eligible to vote
            throw new ArithmeticException("Person is not eligible to vote");
        }
        else {
            System.out.println("Person is eligible to vote!!");
        }
    }
    //main method
    public static void main(String args[]){
        //calling the function
        validate(13);
        System.out.println("rest of the code...");
   }

}
```

Throwing Checked Exception

- Every subclass of Error and RuntimeException is an unchecked exception in Java
- A checked exception is everything else under the Throwable class

Throws clause

- Checked exception (compile time) force you to handle them, if you don’t handle them then the program will not compile
- On the other hand unchecked exception (Runtime) doesn’t get checked during compilation
- Throws keyword is used for handling checked exceptions
- By using throws we can declare multiple exceptions in one go

What is the need of having throws keyword when you can handle exception using try-catch?

- several methods that can cause exceptions, in that case it would be tedious to write these try-catch for each method
- The code will become unnecessary long and will be less-readable
- Overcome this problem is by using throws like this: declare the exceptions in the method signature using throws and handle the exceptions where you are calling this method by using try-catch
- Another advantage of using this approach is that you will be forced to handle the exception when you call this method, all the exceptions that are declared using throws, must be handled where you are calling this method else you will get compilation error

Example of throws Keyword

- In this example the method myMethod() is throwing two checked exceptions so we have declared these exceptions in the method signature using throws Keyword
- If we do not declare these exceptions then the program will throw a compilation error

Syntax of Java throws

```
return_type method_name() throws exception_class_name
{
//method code
}
```

```
import java.io.*;
class Example1 {
     void myMethod(int num)throws IOException,
ClassNotFoundException
  {
    if(num==1)
       throw new IOException("IOException
Occurred");
    else
       throw new
ClassNotFoundException("ClassNotFoundException"
);
  }
}
```

```
public class ThrowsExample1{
  public static void main(String args[]){
   try{
    Example1 obj=new Example1();
     obj.myMethod(1);
   }catch(Exception ex){
    System.out.println(ex);
    }
  }
}
```

O/P: java.io.IOException: IOException Occurred

```
import java.io.*;
public class ThrowChecked {
        //function to check if person is eligible to vote or not
    public static void method() throws
FileNotFoundException {
         FileReader file = new
FileReader("C:\\Users\\Arti\\Desktop\\abc.txt");
       BufferedReader fileInput = new BufferedReader(file);
          throw new FileNotFoundException();
         }
    //main method
 public static void main(String args[]){
       try
       {
          method();
       }
       catch (FileNotFoundException e)
       {
          e.printStackTrace();
       }
       System.out.println("rest of the code...");
  }
}
```

Throws keyword

Rule: If we are calling a method that declares an exception, we must either caught or declare the exception.
There are two cases:
Case 1: We have caught the exception i.e. we have handled the exception using try/catch block
Case 2: We have declared the exception i.e. specified throws keyword with the method

Throws keyword examples

Case 1: Handle Exception Using try-catch block

In case we handle the exception, the code will be executed fine whether exception occurs during the program or not

Case 2: Declare Exception

- In case we declare the exception, if exception does not occur, the code will be executed fine
- In case we declare the exception and the exception occurs, it will be thrown at runtime because throws does not handle the exception

```
import java.io.*;
class ThrowsExample4 {
 void method()throws IOException{
  throw new IOException("device error");
 }
}
 public class Testthrows2{
   public static void main(String args[]){
    try{
     ThrowsExample4 m=new ThrowsExample4();
     m.method();
    }catch(Exception e){System.out.println("exception
handled");}

    System.out.println("normal flow...");
  }
}
```

```
import java.io.*;
public class ThrowsExample5 {
 void method()throws IOException{
  System.out.println("device operation performed");
 }
}
public class Testthrows3{
   public static void main(String args[])throws
IOException{//declare exception
     ThrowsExample5 m=new ThrowsExample5();
     m.method();

    System.out.println("normal flow...");
  }
}
```

```
import java.io.*;
class ThrowsExample6 {

 void method()throws IOException{
  throw new IOException("device error");
 }
}
public class Testthrows4{
   public static void main(String args[])throws
IOException{//declare exception
     ThrowsExample6 m=new ThrowsExample6();
     m.method();

    System.out.println("normal flow...");
  }
}
```

Sr. no.	Basis of Differences	throw	throws
1.	Definition	Java throw keyword is used to throw an exception explicitly in the code, inside the function or the block of code.	Java throws keyword is used in the method signature to declare an exception which might be thrown by the function while the execution of the code.
2.	Type of exception	Using throw keyword, we can only propagate unchecked exception i.e., the checked exception cannot be propagated using throw only.	Using throws keyword, we can declare both checked and unchecked exceptions. However, the throws keyword can be used to propagate checked exceptions only.
3.	Syntax	The throw keyword is followed by an instance of Exception to be thrown.	The throws keyword is followed by class names of Exceptions to be thrown.
4.	Declaration	throw is used within the method.	throws is used with the method signature.
5.	Internal implementation	We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.

User Defined or Custom Exception

- In java we have already defined, exception classes such as ArithmeticException, NullPointerException etc.
- These exceptions are already set to trigger on pre-defined conditions such as when you divide a number by zero it triggers ArithmeticException
- In java we can create our own exception class and throw that exception using throw keyword. These exceptions are known as user-defined or custom exceptions

Why use custom exceptions?

- Java exceptions cover almost all the general type of exceptions that may occur in the programming. However, we sometimes need to create custom exceptions.
- Following are few of the reasons to use custom exceptions:
- To catch and provide specific treatment to a subset of existing Java exceptions.
- Business logic exceptions: These are the exceptions related to business logic and workflow. It is useful for the application users or the developers to understand the exact problem.
- In order to create custom exception, we need to extend Exception class that belongs to java.lang package.

Example Custom Exception


```
public class WrongFileNameException extends Exception
 {
    public WrongFileNameException(String errorMessage)
    {
    super(errorMessage);
    }
}
```

Note: We need to write the constructor that takes the String as the error message and it is called parent class constructor.

Define a class that extends Exception class


```
public class Main{
    public static void main(String[] args) {
        int age=15;
        try{
            if(age<18)
                throw new AgeException("Minor not allowed for voting");
        }catch(AgeException e){
            e.printStackTrace();
        }
    }
}
class AgeException extends Exception{
   AgeException(String message){
      super(message);
   }
}
```

toString() method and getmessage()

- If you want to represent any object as a string, toString() method comes into existence
- The toString() method returns the String representation of the object
- Using the custom exception, we can have your own exception and message
- Here, we have to pass a string to the constructor of superclass i.e. Exception class that can be obtained using getMessage() method on the object we have created.

toString() example


```
class Student{
 int rollno;
 String name;
 String city;

 Student(int rollno, String name, String city){
 this.rollno=rollno;
 this.name=name;
 this.city=city;
 }

 public static void main(String args[]){
   Student s1=new Student(101,"Raj","lucknow");
   Student s2=new Student(102,"Vijay","ghaziabad");

   System.out.println(s1);//compiler writes here s1.toString()
   System.out.println(s2);//compiler writes here s2.toString()
 }
}
```

O/P: Student@1c655221
Student@ee7d9f1

```
class StudentToString {
 int rollno;
 String name;
 String city;
StudentToString (int rollno, String name, String city){
 this.rollno=rollno;
 this.name=name;
 this.city=city;
 }
 public String toString(){//overriding the toString() method
  return rollno+" "+name+" "+city;
 }
 public static void main(String args[]){
   StudentToString  s1=new StudentToString
(101,"Raj","lucknow");
   StudentToString  s2=new StudentToString
(102,"Vijay","ghaziabad");
System.out.println(s1);//compiler writes
here s1.toString()
   System.out.println(s2);//compiler writes
here s2.toString()
 }
}
```

O/P: 101 Raj lucknow
102 Vijay ghaziabad

Example of User defined exception in Java

You can see that while throwing custom exception I gave a string in parenthesis (throw new MyException("This is My error Message");). That’s why we have a parameterized constructor (with a String parameter) in my custom exception class

Note:

1. User-defined exception must extend Exception class
2. The exception is thrown using throw keyword

```
class CustomException extends Exception{
   String str1;
CustomException(String str2) {
                str1=str2;
   }
   public String toString(){
                return ("MyException Occurred: "+str1) ;
   }
}

class Example1{
   public static void main(String args[]){
                try{
                                System.out.println("Starting of try
block");
                                // I'm throwing the custom
exception using throw
                                throw new CustomException("This
is My error Message");
                }
catch(CustomException exp){

                                System.out.println("Catch Block") ;
                                                System.out.println(exp)
;
                } } }
```

O/P : Starting of try block
Catch Block
MyException Occurred: This is My error Message

User Defined Exception Example

In this example we are throwing an exception from a method
In this case we should use throws clause in the method signature otherwise you will get compilation error saying that “unhandled exception in method”

```
class CustomExceptionExample2 extends Exception
{
    public CustomExceptionExample2(String s)
    {
        // Call constructor of parent Exception
        super(s);
    }
}

 class Example
{
   void productCheck(int weight) throws
CustomExceptionExample2{
                if(weight<100){
                                throw new
CustomExceptionExample2("Product Invalid");
                }
   }
```

```
public static void main(String args[])
    {
                Example obj = new Example();
        try
        {
            obj.productCheck(60);
        }
        catch (CustomExceptionExample2 ex)
        {
            System.out.println("Caught the
exception");
            System.out.println(ex.getMessage());
        }
    }
}
```

User Defined Exception Example

- In the following code, constructor of InvalidAgeException takes a string as an argument
- This string is passed to constructor of parent class Exception using the super() method
- Also the constructor of Exception class can be called without using a parameter and calling super() method is not mandatory

```
public class CustomExceptionExample3 extends Exception {
    // class representing custom exception

    public CustomExceptionExample3 (String str)
    {
        // calling the constructor of parent Exception
        super(str);
    }
}

// class that uses custom exception InvalidAgeException
 class TestCustomException1
{

    // method to check the age
    static void validate (int age) throws CustomExceptionExample3{
        if(age < 18){

        // throw an object of user defined exception
        throw new CustomExceptionExample3("age is not valid to vote");
    }
        else {
        System.out.println("welcome to vote");
        }
     }
```

```
 // main method
    public static void main(String args[])
    {
        try
        {
            // calling the method
            validate(13);
        }
        catch (CustomExceptionExample3 ex)
        {
            System.out.println("Caught the exception");

            // printing the message from
InvalidAgeException object
            System.out.println("Exception occured: " +
ex);
        }

        System.out.println("rest of the code...");
    }
}
```

Exercise

- WAP which takes 3 sides of a triangle as input and finds area of a triangle.
- A triangle is valid if it satisfies triangle inequality theorem.
- If a triangle is found invalid, throw an InvalidTriangleException
- Define appropriate handler code to handle the exception.

Predict the output


```
class Main {
   public static void main(String args[]) {
      try {
         throw 10;
      }
      catch(int e) {
         System.out.println("Got the  Exception " + e);
      }
  }
}
```

(A) Got the Exception 10
(B) Got the Exception 0
(C) Compiler Error

Write a java program to create own exception for Negative Value Exception if the user enter negative value.

Write a Java program that asks the user for a number and displays ten times the number. Also the program must throw an exception when the user enters a value greater than 10.

```
class NegativeValueException extends Exception
 {
    public NegativeValueException(String message)
 {
        super(message);
    }
}

class UserInput {
    public static void main(String[] args) {
        try {
            int userInput = getUserInput();
            if (userInput < 0) {
                throw new NegativeValueException("Negative values are not allowed!");
            }
            System.out.println("User input: " + userInput);
        } catch (NegativeValueException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }


    public static int getUserInput() throws NegativeValueException {
        // Simulating user input
        int userInput = -5; // Change this value to test different inputs

        if (userInput < 0) {
            throw new NegativeValueException("Negative values are not allowed!");
        }

        return userInput;
    }
}
```

```
import java.util.Scanner;

public class NumberMultiplier {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            System.out.print("Enter a number: ");
            int number = scanner.nextInt();

            if (number > 10) {
                throw new Exception("Number cannot be greater than 10");
            }

            int result = number * 10;
            System.out.println("Ten times the number is: " + result);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

Java I/O

Java I/O (Input and Output) is used *to process the input* and *produce the output*
Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations
We can perform **file handling in Java** by Java I/O API

Stream

- Stream is an abstraction that either produces or consumes information
- Stream is a flow /sequence of data
- It is the channel through which data is input or output in a java program
- An input stream can abstract many different kinds of input: from a disk file, a keyboard, or a network socket
- Likewise, an output stream may refer to the console, a disk file, or a network connection

Why is Stream an Abstraction?

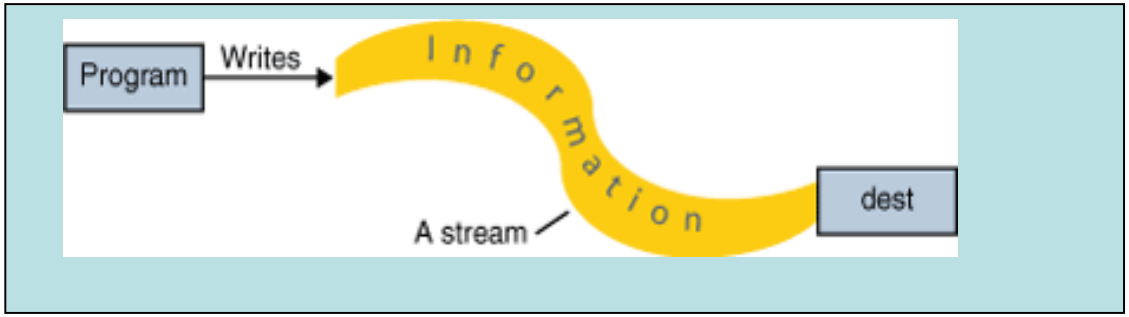
- All streams behave in the same manner, even if the actual physical devices to which they are linked differ
- Thus, the same I/O classes and methods can be applied to different types of devices
- This means that an input/output stream can abstract many different kinds of input/output

Overview of I/O Streams

To **bring/read** in information, a program opens a *stream* on an information source (a file, memory, a socket) and reads the information sequentially, as shown in the following figure.

Overview of I/O Streams

Similarly, a program can send/write information to an external destination by opening a stream to a destination and writing the information out sequentially, as shown in the following figure.



Overview of I/O streams

- The java.io package contains a collection of stream classes that support algorithms for reading and writing
- To use these classes, a program needs to import the java.io package
- The stream classes are divided into two class hierarchies, based on the data type (either characters or bytes) on which they operate i.e

Character Stream
and
Byte Stream

Character Streams

- **Reader and Writer** are the abstract superclasses for character streams in java.io
- Reader provide streams **that read 16-bit characters** and Writer provides the streams that **write 16-bit characters**
- Character streams are more efficient than byte streams

Byte Streams

- To read and write **8-bit bytes**, programs should use the byte streams, descendants of InputStream and OutputStream
- InputStream and OutputStream provide streams that read 8-bit bytes and write 8-bit bytes
- Byte streams are used, for example, when reading or writing binary data such as images and audio

Byte Streams

- Byte streams can deal with transfer of ASCII characters ranging 0 to 255
- JAVA supports Unicode characters that are 16 bits long
- Character streams support Unicode characters

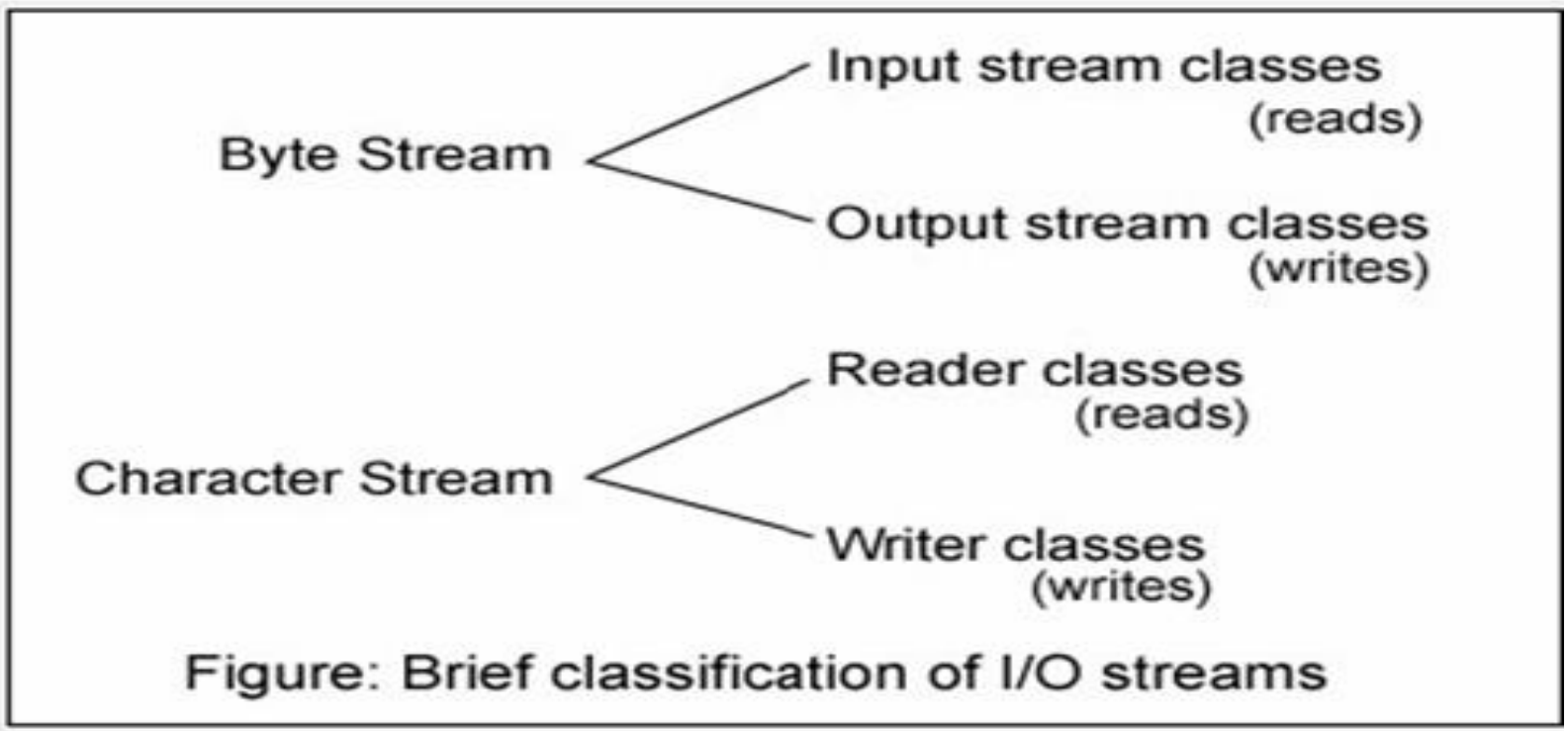
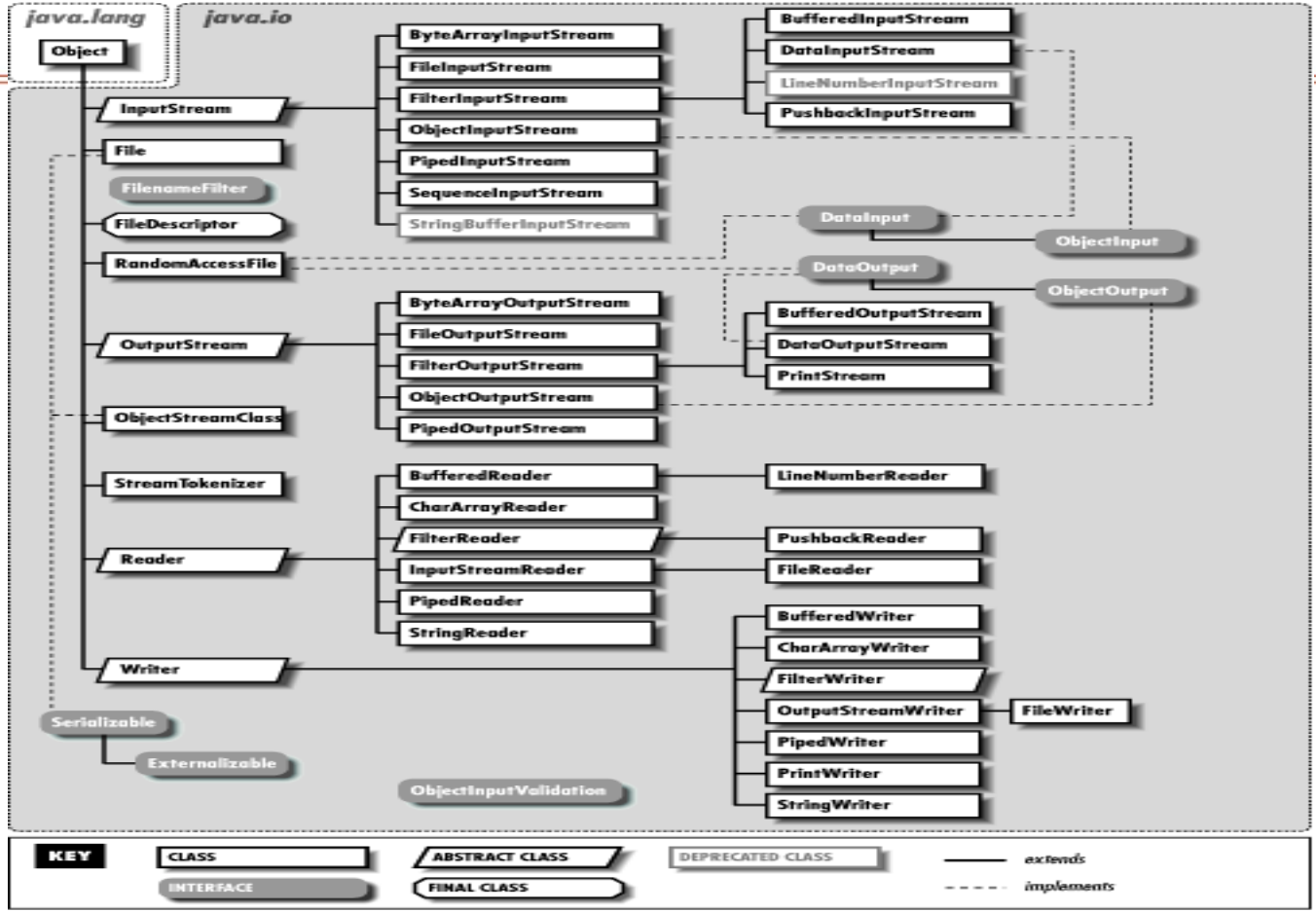
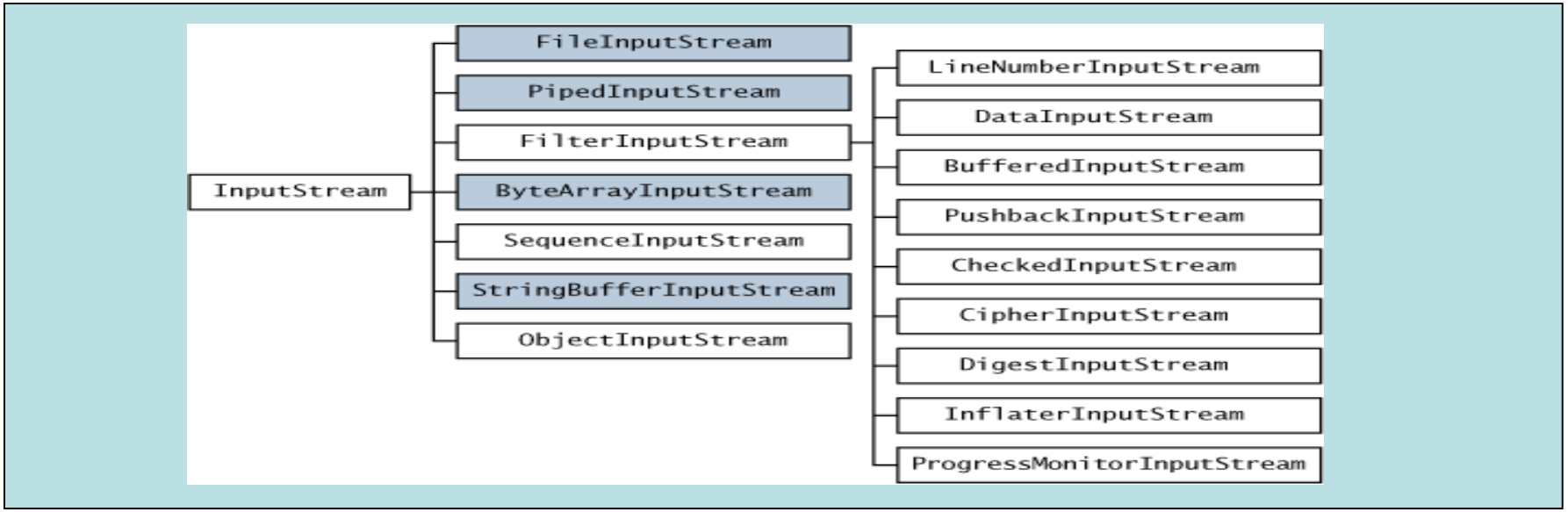


Figure: Brief classification of I/O streams


java.lang
java.io
Object
InputStream
File
FilenameFilter
FileDescriptor
RandomAccessFile
OutputStream
ObjectStreamClass
StreamTokenizer
Reader
Writer
Serializable
Externalizable
ObjectInputValidation
ByteArrayInputStream
FileInputStream
FilterInputStream
ObjectInputStream
PipedInputStream
SequenceInputStream
StringBufferInputStream
BufferedInputStream
DataInputStream
LineNumberInputStream
PushbackInputStream
DataInput
ObjectInput
DataOutput
ObjectOutput
ByteArrayOutputStream
FileOutputStream
FilterOutputStream
ObjectOutputStream
PipedOutputStream
BufferedOutputStream
DataOutputStream
PrintStream
BufferedReader
CharArrayReader
FilterReader
InputStreamReader
PipedReader
StringReader
LineNumberReader
PushbackReader
FileReader
BufferedWriter
CharArrayWriter
FilterWriter
OutputStreamWriter
PipedWriter
PrintWriter
StringWriter
FileWriter
KEY
CLASS
ABSTRACT CLASS
DEPRECATED CLASS
INTERFACE
FINAL CLASS
extends
implements

Byte Streams (cont.)

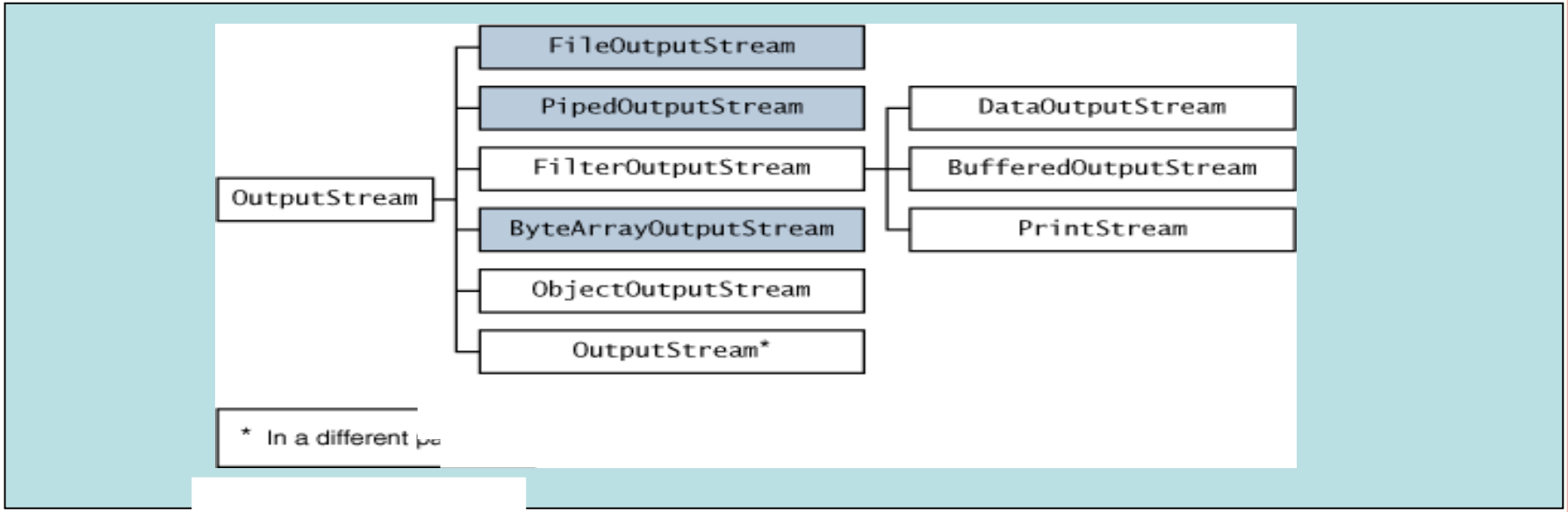
The class hierarchy for the InputStream Class



Byte Stream (cont.)

RCOEM
Shri Ramdeobaba College of Engineering and Management, Nagpur

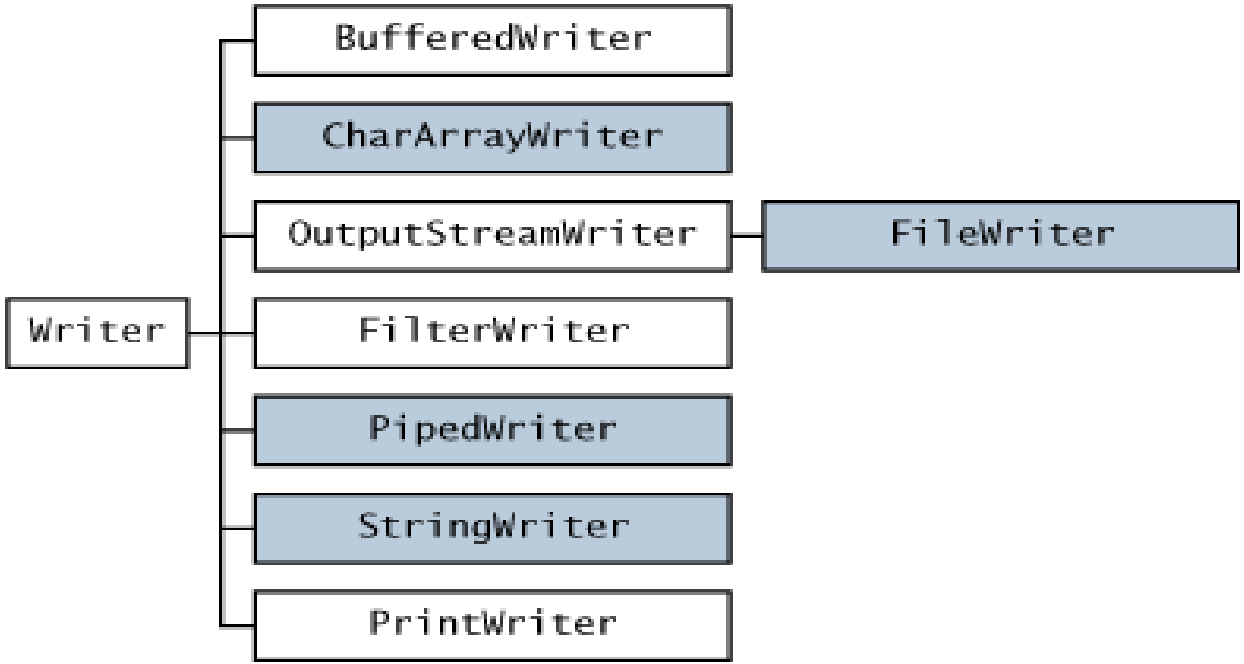
Class hierarchy for Output Class



Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements **InputStream**
FilterOutputStream	Implements **OutputStream**
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains **print( )** and **println( )**
PushbackInputStream	Input stream that supports one-byte "unget," which returns a byte to the input stream
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

Character Streams Contd.

The following figure shows the class hierarchies for the Reader and Writer classes



Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains **print( )** and **println( )**
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

Standard Streams

- JAVA provides support for standard I/O i.e. keyboard and console
- System is a class in java.lang package
- in, out and err are public static and final variables in System class
- in- object of InputStream
- out, err – objects of PrintStream
- Three predefined streams are available to a program
- System.in – to read input from keyboard
- System.out – to output the data produced by user program to console
- System.err – to output the error data produced by user program to console

Reading and Writing Files

Byte oriented File I/O

– FileInputStream

– FileOutputStream

Character oriented File I/O

– FileReader

– FileWriter

Using FileInputStream

FileInputStream(String *fileName*) throws FileNotFoundException
FileOutputStream(String *fileName*) throws FileNotFoundException
Used to read from file

FileInputStream f = new FileInputStream ("C:/java/hello.txt");

File f=new File("C:/java/hello");

FileInputStream f=new FileInputStream(f);

FileInputStream methods

public void close() throws IOException{}

This method closes the file input stream. Releases any system resources associated with the file. Throws an IOException

public int read() throws IOException{}

Reads the integer representation of next available byte. On end of file returns -1

public int read(byte[] r) throws IOException{}

This method reads r length bytes from the input stream into an array. Returns the total number of bytes read. If end of file -1 will be returned

Closing a File

- When you are done with a file, you must close it
- Done by calling the **close( )** method, which is implemented by both **FileInputStream** and **FileOutputStream**

  void close( ) throws IOException
- Closing a file releases the system resources allocated to the file, allowing them to be used by another file
- Failure to close a file can result in “memory leaks” because of unused resources remaining allocated
- There are two ways to close a file
- close() method
- Try with resources

Sample Program

WAP to display the contents of a file that contains ASCII text

Using FileOutputStream

Used to create a file and write into it

```
FileOutputStream f = new FileOutputStream("C:/java/hello")
```

```
File f = new File("C:/java/hello");
FileOutputStream f = new FileOutputStream(f);
```

FileOutputStream methods

public void close() throws IOException{}
This method closes the file output stream. Releases any system resources associated with the file. Throws an IOException

public void write(int w)throws IOException{}
This methods writes the specified byte to the output stream. Although *w* is declared as an integer, only the low-order eight bits are written to the file

public void write(byte[] w)
Writes w length bytes from the byte array to the OutputStream

Sample Program

WAP to copy a file called FIRST.TXT to a file called SECOND.TXT

Automatically Closing a File : Automatic Resource Management (ARM)

Use try with resource

try (*resource-specification*) {
    // use the resource
}

Here, *resource-specification* is a statement that **declares** and **initializes** a resource

When the **try** block ends, the resource is automatically released

Thus, there is no need to call **close( )** explicitly

The **try**-with-resources statement can be used only with those resources that implement the **AutoCloseable** interface defined by **java.lang**

Example

The following code uses a try-with-resources statement to open a file and then automatically close it when the try block is left

```
try(FileInputStream fin = new FileInputStream(args[0])) {
   do {
        i = fin.read();
        if(i != -1)
            System.out.print((char) i);
   }while(i != -1);
}catch(IOException e) {
   System.out.println("File Not Found.");
}
```

Try with resource

- The resource declared in the **try** statement is implicitly **final**
- This means that you can't assign to the resource after it has been created. Also, the scope of the resource is limited to the **try**-with-resources statement
- You can manage more than one resource within a single **try** statement. To do so, simply separate each resource specification with a semicolon
- ***try (FileInputStream fin = new FileInputStream(args[0]);***
- ***FileOutputStream fout = new FileOutputStream(args[1]))***

Java FileReader Class

RCOEM
Shri Ramdeobaba College of Engineering and Management, Nagpur

- Java FileReader class is used to read data from the file
- It returns data in byte format like FileInputStream class
- It is character-oriented class which is used for file handling in Java
- Java FileReader class declaration
- public class FileReader extends InputStreamReader

Constructors of FileReader class

RCOEM
Shri Ramdeobaba College of Engineering and Management, Nagpur

Constructor	Description
FileReader(String file)	It gets filename in string. It opens the given file in read mode. If file doesn't exist, it throws FileNotFoundException.
FileReader(File file)	It gets filename in file instance. It opens the given file in read mode. If file doesn't exist, it throws FileNotFoundException.

Methods of FileReader class

Method	Description
int read()	It is used to return a character in ASCII form. It returns -1 at the end of file.
void close()	It is used to close the FileReader class.

Java FileReader Example

In this example, we are reading the data from the text file **testout.txt** using Java FileReader class

```
package filereader;
import java.io.FileReader;
public class FileReaderExample1 {

    public static void main(String args[])throws Exception{
        FileReader fr=new FileReader("D:\\testout2.txt");
        int i;
        while((i=fr.read())!=-1)
        System.out.print((char)i);
        fr.close();
    }
}
```

File Writer

- Java FileWriter class is used to write character-oriented data to a file
- It is character-oriented class which is used for file handling in java
- Unlike FileOutputStream class, you don't need to convert string into byte array because it provides method to write string directly
- Java FileWriter class declaration
- public class FileWriter extends OutputStreamWriter

Constructors of FileWriter class

RCOEM
Shri Ramdeobaba College of Engineering and Management, Nagpur

Constructor	Description
FileWriter(String file)	Creates a new file. It gets file name in string.
FileWriter(File file)	Creates a new file. It gets file name in File object.

Methods of FileWriter class

Method	Description
void write(String text)	It is used to write the string into FileWriter.
void write(char c)	It is used to write the char into FileWriter.
void write(char[] c)	It is used to write char array into FileWriter.
void flush()	It is used to flushes the data of FileWriter.
void close()	It is used to close the FileWriter.

Java FileWriter Example

In this example, we are writing the data in the file testout.txt using Java FileWriter class.

```
package filewriter;
import java.io.FileWriter;
public class FileWriterExample1 {

    public static void main(String args[]){
        try{
          FileWriter fw=new FileWriter("D:\\testout.txt");
          fw.write("Hello World");
          fw.close();
         }catch(Exception e){System.out.println(e);}
         System.out.println("Success...");
     }
}
```

Java BufferedWriter Class

- Java BufferedWriter class is used to provide buffering for Writer instances. It makes the performance fast
- It inherits Writer class
- The buffering characters are used for providing the efficient writing of single arrays, characters, and strings
- Class declaration
- public class BufferedWriter extends Writer

Class constructors

Constructor	Description
BufferedWriter(Writer wrt)	It is used to create a buffered character output stream that uses the default size for an output buffer.
BufferedWriter(Writer wrt, int size)	It is used to create a buffered character output stream that uses the specified size for an output buffer.

Class methods

Method	Description
void newLine()	It is used to add a new line by writing a line separator.
void write(int c)	It is used to write a single character.
void write(char[] cbuf, int off, int len)	It is used to write a portion of an array of characters.
void write(String s, int off, int len)	It is used to write a portion of a string.
void flush()	It is used to flushes the input stream.
void close()	It is used to closes the input stream

Example of Java BufferedWriter

Let's see the simple example of writing the data to a text file **testout.txt** using Java BufferedWriter

```
package bufferedwriter;
import java.io.*;
public class BufferedWriterEample {

public static void main(String[] args) throws Exception {
    FileWriter writer = new FileWriter("D:\\testout.txt");
    BufferedWriter buffer = new BufferedWriter(writer);
    buffer.write("Hello World");
    buffer.close();
    System.out.println("Success");
    }
}
```

Java BufferedReader Class

- Java BufferedReader class is used to read the text from a character-based input stream
- It can be used to read data line by line by readLine() method
- It makes the performance fast
- It inherits Reader class
- Java BufferedReader class declaration
- public class BufferedReader extends Reader

Java BufferedReader class constructors

Constructor	Description
BufferedReader(Reader rd)	It is used to create a buffered character input stream that uses the default size for an input buffer.
BufferedReader(Reader rd, int size)	It is used to create a buffered character input stream that uses the specified size for an input buffer.

Java BufferedReader class methods

Method	Description
int read()	It is used for reading a single character.
int read(char[] cbuf, int off, int len)	It is used for reading characters into a portion of an array.
boolean markSupported()	It is used to test the input stream support for the mark and reset method.
String readLine()	It is used for reading a line of text.
boolean ready()	It is used to test whether the input stream is ready to be read.
long skip(long n)	It is used for skipping the characters.
void reset()	It repositions the stream at a position the mark method was last called on this input stream.
void mark(int readAheadLimit)	It is used for marking the present position in a stream.
void close()	It closes the input stream and releases any of the system resources associated with the stream.

Java BufferedReader Example

In this example, we are reading the data from the text file **testout.txt** using Java BufferedReader class

```
import java.io.*;
public class BufferedReaderExample {
    public static void main(String args[])throws Exception{
        FileReader fr=new FileReader("D:\\testout.txt");
        BufferedReader br=new BufferedReader(fr);

        int i;
        while((i=br.read())!=-1){
        System.out.print((char)i);
        }
        br.close();
        fr.close();
    }
}
```

Reading input from Console

There are 3 ways :

1) InputStreamReader wrapped in a BufferedReader
2) Scanner class
3) Console class

Method 1

1) Create an **InputStreamReader with System.in**

2) Create **BufferedReader using InputStreamReader**

3) Get user input by calling BufferedReader methods like, readLine() and read()

InputStreamReader

- InputStreamReader is a **bridge** between byte stream and character stream
- It **reads bytes and decodes into character**
- **Constructor** takes an object of **InputStream**

Scanner

-Class defined in util package
-Usage

- Scanner scanIn = new Scanner (System.in);

-Methods like

- nextLine() - reads line as a string
- nextInt() - reads integer

Using Console class


```
Import java.io.Console;
Console console = System.console();
String input = console.readLine();
```

How to Use Pipe Streams

Pipes are used to **channel the output from one thread into the input of another.** PipedReader and PipedWriter (and their input and output stream counterparts PipedInputStream and PipedOutputStream) implement the input and output components of a pipe

How to wrap a stream

Streams are wrapped to combine the various features of the many streams.

Example code:

```
BufferedReader in = new BufferedReader(source);
```

The code opens a BufferedReader on source, which is another reader of a different type. This essentially "wraps" source in a BufferedReader. The program reads from the BufferedReader, which in turn reads from source.

Working with Filter Streams

- The java.io package provides a set of abstract classes that define and partially implement *filter streams*. A filter stream filters data as it's being read from or written to the stream.
- The filter streams are FilterInputStream , and FilterOutputStream .
- A filter stream is constructed on another stream (the *underlying* stream).

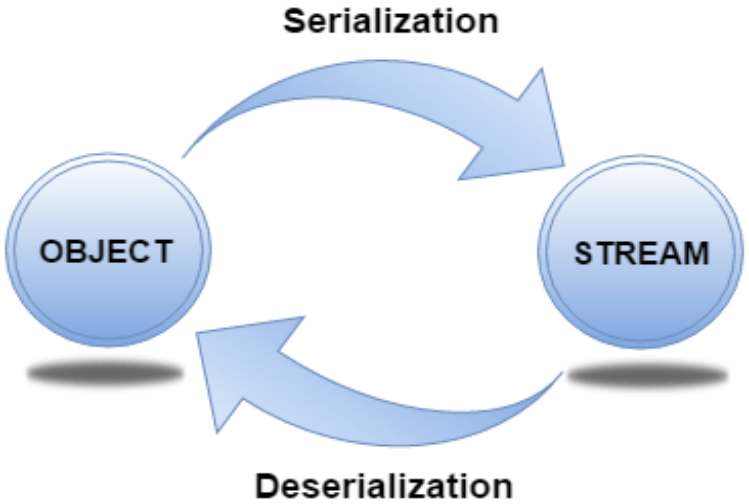
Working with Random Access Files

- A *random access file* permits non-sequential or random access to a file's contents.
- **Using Random Access Files**
- Unlike the input and output stream classes in java.io, RandomAccessFile is used for both reading and writing files. You create a RandomAccessFile object with different arguments depending on whether you intend to read or write.

Serialization and Deserialization

- **Serialization in Java** is a mechanism of *writing the state of an object into a byte-stream*
- The reverse operation of serialization is called *deserialization* where byte-stream is converted into an object
- The serialization and deserialization process is platform-independent, it means you can serialize an object on one platform and deserialize it on a different platform
- For serializing the object, we call the **writeObject()** method of *ObjectOutputStream* class, and for deserialization we call the **readObject()** method of *ObjectInputStream* class
- We must have to implement the *Serializable* interface for serializing the object

Serialization and Deserialization



java.io.Serializable interface

- **Serializable** is a marker interface (has no data member and method). It is used to "mark" Java classes so that the objects of these classes may get a certain capability. The **Cloneable** and **Remote** are also marker interfaces.
- The **Serializable** interface must be implemented by the class whose object needs to be persisted.
- The String class and all the wrapper classes implement the *java.io.Serializable* interface by default.

Example


```
import java.io.Serializable;
public class Student implements Serializable{
 int id;
 String name;
 public Student(int id, String name) {
  this.id = id;
  this.name = name;
 }
}
```

In the above example, ***Student*** class implements Serializable interface. Now its objects can be converted into stream.

ObjectOutputStream class

- The ObjectOutputStream class is used to write primitive data types, and Java objects to an OutputStream
- Only objects that support the java.io.Serializable interface can be written to streams
-

Constructor

public ObjectOutputStream(OutputStream out) throws IOException {}	It creates an ObjectOutputStream that writes to the specified OutputStream.

Important Methods

Method	Description
1) public final void writeObject(Object obj) throws IOException {}	It writes the specified object to the ObjectOutputStream.
2) public void flush() throws IOException {}	It flushes the current output stream.
3) public void close() throws IOException {}	It closes the current output stream.

ObjectInputStream class

An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

Constructor

1) public ObjectInputStream(InputStream in) throws IOException {}	**It creates an ObjectInputStream that reads from the specified InputStream.**

Important Methods

Method	Description
1) public final Object readObject() throws IOException, ClassNotFoundException{}	It reads an object from the input stream.
2) public void close() throws IOException {}	It closes ObjectInputStream.

Example of Java Serialization

- In this example, we are going to serialize the object of ***Student*** class
- The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object
- We are saving the state of the object in the file named f.txt.

```
import java.io.Serializable;
public class Student implements Serializable{
 int id;
 String name;
 public Student(int id, String name) {
  this.id = id;
  this.name = name;
 }
}
```

Serialization example:


```
import java.io.*;
class Persist{
 public static void main(String args[]){
  try{
  //Creating the object
  Student s1 =new Student(211,"ravi");
  //Creating stream and writing the object
  FileOutputStream fout=new FileOutputStream("f.txt");
  ObjectOutputStream out=new ObjectOutputStream(fout);
  out.writeObject(s1);
  out.flush();
  //closing the stream
  out.close();
  System.out.println("success");
  }catch(Exception e){System.out.println(e);}
 }
}
```

Output:
Success

Example of Java Deserialization

- Deserialization is the process of reconstructing the object from the serialized state
- It is the reverse operation of serialization
- Let's see an example where we are reading the data from a deserialized object

Example of Java Deserialization


```
import java.io.*;
public class DeserializeDemo
{
public static void main(String [] args)
 {
Employee e = null;
Try
 {
FileInputStream fileIn = new FileInputStream("/tmp/employee.ser");
ObjectInputStream in = new ObjectInputStream(fileIn);
e = (Employee) in.readObject();
In.close();
 fileIn.close();
}
catch (IOException i)
{
 i.printStackTrace();
return;
}
catch (ClassNotFoundException c)
{
System.out.println("Employee class not found");
c.printStackTrace();
return;
}
 System.out.println("Deserialized Employee...");
System.out.println("Name: " + e.name);
 System.out.println("Address: " + e.address);
System.out.println("SSN: " + e.SSN);
System.out.println("Number: " + e.number); }
}
```

Output:

Deserialized Employee... Name: Reyan Ali Address:Phokka Kuan, Ambehta Peer SSN: 0 Number:10